MLP Tic-Tac-Toe Web Application

Team Member: Ken, Nitipat Wuttisasiwat **CWID:** 867656993

Problem Statement

The goal of this project is to create a web-based Tic-Tac-Toe game where the user plays as 'X' and an AI agent plays as 'O'. The AI agent learns over time using reinforcement learning, improving its performance after each game.

Approach

To make the AI agent learn effectively, we apply Deep Q-Learning using a Multi-Layer Perceptron (MLP) model. For simplicity and consistency, the player always starts first as 'X' and the agent plays second as 'O'. The MLP architecture is composed of three layers:

- **Input Layer:** 9 units, each representing a cell in the 3x3 Tic-Tac-Toe board with possible 3 values (‘X’, ‘O’, null).
- **Hidden Layers:** Two fully connected layers with 64 units each, using ReLU activation.
- **Output Layer:** 9 units with softmax activation, representing predicted Q-values for all possible moves.

The model is trained using stochastic gradient descent with the Adam optimizer and mean squared error loss.

To choose actions, the agent uses a greedy strategy, selecting the move with the highest predicted Q-value among all valid moves. We intentionally skip introducing stochasticity (e.g., ε-greedy or discount factor γ) to keep the behavior deterministic and the implementation simple.

During training, we update the model by looping through game memories, adjusting Q-values based on immediate rewards, and fitting the model one step at a time. Although simplified, this method helps the agent avoid past mistakes after just a few games.

Description of the Software

- **Frontend/UI:** Built with React.js and styled using TailwindCSS and Shadcn component library. The interface allows users to interact with the game, reset progress
- **Algorithm:** Implemented in **TypeScript** using **TensorFlow.js** for neural network modeling and training.

Evaluation

To evaluate the learning performance of the AI agent, we conducted multiple play sessions and monitored behavioral changes over time. The evaluation focused on both qualitative observations and simple quantitative tracking.

- **Learning Behavior:**
  After just 5 to 10 rounds of play, the AI began avoiding common losing strategies, such as placing 'O' into positions that allow the opponent to create forks or win in the next turn. This suggests that even with simplified Deep Q-Learning, the model can effectively reinforce winning behavior based on past outcomes.
- **Greedy Strategy Outcome:**
  Since the agent uses a purely greedy policy (no exploration), it quickly converges to favoring specific high-reward moves based on initial game experiences. This results in rapid but potentially narrow learning — it can overfit to early strategies but lacks broader tactical awareness.
- **Win/Loss Tracking:**
  While we did not implement detailed analytics, we observed a noticeable increase in the number of draws or AI wins after the agent had trained for around 15 games. Future versions could incorporate exact win/loss/draw counters to quantify progress more accurately.
- **Training Efficiency:**
  The agent trains incrementally after each game using in-browser TensorFlow.js, with minimal performance overhead. Training is lightweight (1 epoch per game), making it suitable for real-time, continuous learning during user interaction.

Conclusion and Future Work

This project demonstrated that Deep Q-Learning with an MLP is an overpowered yet educational approach for solving a simple game like Tic-Tac-Toe. It was a valuable hands-on experience with tools like TensorFlow.js and React.

Potential Improvements:

- Visualizing the full MLP graphically, including per-layer neuron values and weight heatmaps.
- Introducing more advanced exploration strategies like ε-greedy.
- Saving longer-term game history for more complex training.

References

- **Deployed Application:**
  https://kennaruk.github.io/mlp-tensorflowjs-tictactoe/

- **GitHub Repository (Code + README):**
  https://github.com/kennaruk/mlp-tensorflowjs-tictactoe

- **TensorFlow.js Documentation**: https://js.tensorflow.org/